Relatório de Auditoria de Segurança

AutoRecurso

Data: 31/08/2026

Escopo: Código-fonte completo (frontend React + backend Express + deploy Docker)

Nota Metodológica

Stack detectada: TypeScript • Express 5 (backend) • React 19 + Vite (frontend) • SQLite/better-sqlite3 (banco) • JWT + senha fixa (auth admin) • Docker Compose + Nginx (deploy).

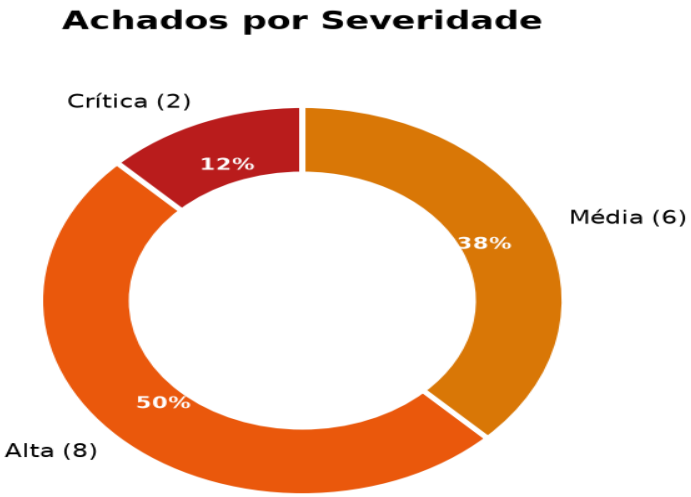
Mapeamento das categorias:

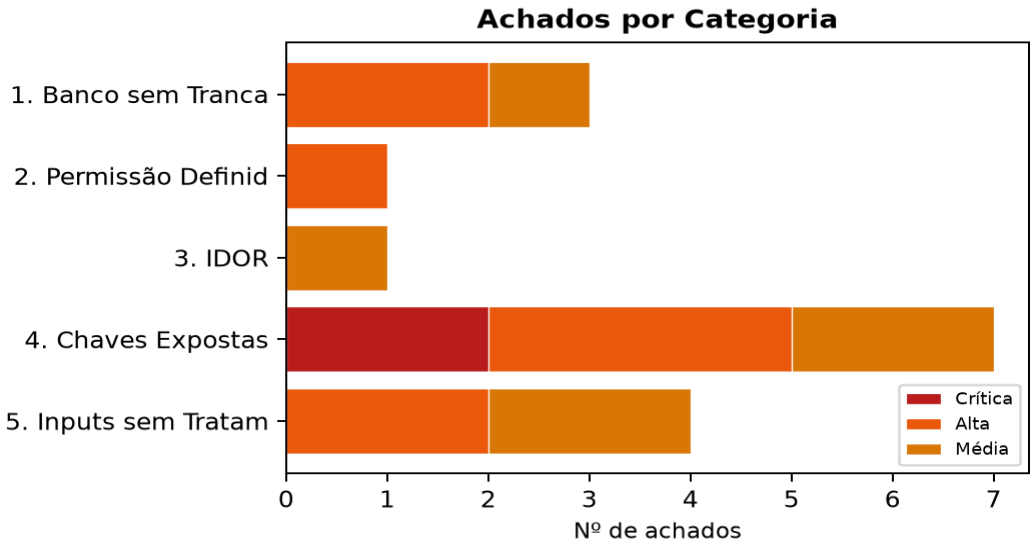
1. *Banco sem Tranca* → Filtro manual por user inexistente (sem RLS, sem ORM com scoping).
2. *Permissão no Navegador* → Gate de admin via token local vs. authMiddleware no backend.
3. *IDOR* → Handlers que operam por ID/email sem verificação de posse.
4. *Chaves Expostas* → .env, .env.example, fallbacks em código, histórico git.
5. *XSS* → Templates de email com interpolação, ReactMarkdown, dangerouslySetInnerHTML.

Resumo Executivo

Foram identificados **16 achados** de segurança e **8 pontos fortes** na análise completa do código-fonte do projeto AutoRecurso.

Severidade	Quantidade
Crítica	2
Alta	8
Média	6
TOTAL	**16**





Pontos Fortes (Controles Verificados)

■ Rotas admin protegidas por authMiddleware (JWT)

As rotas /auto-api/admin/data, /admin/settings e /admin/clear usam corretamente o authMiddleware que valida JWT.

Evidência: server/index.ts:464, 474, 483

■ Arquivos .env no .gitignore

Os arquivos .env e server/.env estão no .gitignore e NÃO aparecem no git ls-files. Impede commit acidental.

Evidência: .gitignore:25

■ CORS habilitado (embora aberto)

O servidor usa cors() — protege contra CSRF simples mas aceita todas as origens.

Evidência: server/index.ts:57

■ Sanitização de HTML no emailService (sendResourceEmail)

A função sendResourceEmail faz escape de &, < e > no conteúdo do recurso antes de inserir no template HTML do email.

Evidência: services/emailService.ts:31

■ Headers de segurança no Nginx

X-Frame-Options SAMEORIGIN, X-Content-Type-Options nosniff e X-XSS-Protection configurados.

Evidência: nginx.conf:38-40

■ Sem dangerouslySetInnerHTML no frontend

Nenhum uso de dangerouslySetInnerHTML, innerHTML direto, eval() ou new Function() foi encontrado nos componentes React.

Evidência: App.tsx, Admin.tsx — verificação completa

■ ReactMarkdown sem rehype-raw

Embora rehype-raw esteja no package.json, NÃO é importado nem usado no ReactMarkdown do Admin.tsx. Assim, HTML cru não é renderizado.

Evidência: Admin.tsx:687

■ SQLite com WAL e busy_timeout

Configuração de concorrência adequada para o porte do projeto.

Evidência: server/db.ts:21-23

Pontos Fracos (Riscos Centrais)

■■ Segredos (JWT, senhas, API keys) com valores default inseguros e sem validação de startup.

■■ Múltiplas rotas sensíveis acessíveis sem autenticação (register-resource, analytics, email/send).

■■ Bypass de pagamento trivial via parâmetro de query ou substring no email.

■■ Dados pessoais (LGPD) expostos via busca sem autenticação.

■■ Templates de email vulneráveis a XSS/phishing por falta de escape nos inputs.

Achados Detalhados por Categoria

1. Banco sem Tranca

[ALTA] Rota register-resource sem autenticação

■ **server/index.ts** — linhas 502-509

POST /auto-api/admin/register-resource NÃO usa authMiddleware. Qualquer pessoa pode inserir recursos falsos no banco.

```
app.post('/auto-api/admin/register-resource', (req, res) => { ... })
```

Impacto: Inserção arbitrária de dados no banco; poluição do dashboard admin.

[ALTA] Rota analytics/event sem autenticação

■ **server/index.ts** — linhas 492-500

POST /auto-api/analytics/event aceita qualquer payload sem token. Um atacante pode injetar eventos falsos (payment_completed, etc.) que inflam métricas de receita e distorcem o dashboard.

```
app.post('/auto-api/analytics/event', (req, res) => { addEvent(req.body); ... })
```

Impacto: Fraude de métricas; o admin vê receita fictícia e toma decisões erradas.

[MÉDIA] Busca de recursos por CPF/placa/email sem autenticação

■ **server/index.ts** — linhas 512-525

POST /auto-api/resource/recover permite buscar recursos de QUALQUER cliente informando CPF, e-mail ou placa. Não há rate-limiting nem verificação de que o chamador é o dono dos dados.

```
app.post('/auto-api/resource/recover', (req, res) => { const { query } = req.body; ... })
```

Impacto: Enumeração de dados pessoais (LGPD). Permite descobrir CPFs, nomes e documentos de outros clientes.

2. Permissão Definida no Navegador

[ALTA] Bypass de pagamento via parâmetro dev=true / email com "test"

■ **server/index.ts** — linhas 396-405

GET /auto-api/payment/verify/:email auto-aprova pagamento se req.query.dev === "true" OU se o email contém "teste"/"test"/"dev". Em produção (APP_MODE != dev), o parâmetro ?dev=true AINDA funciona. Um atacante usa email "teste@x.com" ou ?dev=true para obter PAID sem pagar.

```
const isDev = process.env.APP_MODE === 'dev' || req.query.dev === 'true' ||
email.toLowerCase().includes('teste') || ...
```

Impacto: Bypass completo do fluxo de pagamento; perda de receita direta.

3. IDOR

[MÉDIA] Verificação de pagamento por email sem autenticação

■ **server/index.ts** — linhas 391-452

GET /auto-api/payment/verify/:email consulta status de pagamento de qualquer email. Não requer autenticação. Permite enumerar quais emails já pagaram.

```
app.get('/auto-api/payment/verify/:email', async (req, res) => { ... })
```

Impacto: Exposição de informação de pagamento de terceiros.

4. Chaves Expostas

[CRÍTICA] JWT_SECRET hardcoded com valor default inseguro

■ **server/index.ts** — linhas 97

JWT_SECRET recebe fallback 'supersecretkey_change_me_in_production'. Se a variável de ambiente não for definida (ou for esquecida), TODOS os tokens admin podem ser forjados por qualquer pessoa.

```
const JWT_SECRET = process.env.JWT_SECRET || 'supersecretkey_change_me_in_production';
```

Impacto: Forja de token admin → acesso total ao painel administrativo.

[CRÍTICA] Chaves de API reais nos arquivos .env (no disco do projeto)

■ **.env + server/.env** — linhas 1-12 / 1-9

Ambos os arquivos .env contêm chaves REAIS de produção: GEMINI_API_KEY (AIzaSy…), BREVO_API_KEY (xkeysib-…), KIWIFY_CLIENT_SECRET, ADMIN_PASSWORD=admin123, JWT_SECRET. Embora estejam no .gitignore, qualquer acesso ao servidor expõe tudo. Além disso, o .env.example contém uma chave real do AbacatePay.

```
GEMINI_API_KEY=AIzaSyA3pJn... ADMIN_PASSWORD=admin123 JWT_SECRET=supersecretkey_change_me_in_production
```

Impacto: Comprometimento de todas as APIs externas e acesso admin.

[ALTA] ADMIN_PASSWORD trivial (admin123)

■ **server/.env** — linhas 3

A senha de admin é "admin123", um dos valores mais comuns em ataques de dicionário. Não há proteção contra brute-force no endpoint de login.

```
ADMIN_PASSWORD=admin123
```

Impacto: Login admin trivial por força bruta ou tentativa direta.

[ALTA] Chave AbacatePay real no .env.example (versionado no git)

■ **.env.example** — linhas 4

.env.example está rastreado no git e contém VITE_ABACATE_PAY_API_KEY=abc_dev_kqAuKtPpgq5XgUwRNCggq0BQ. Mesmo sendo "dev", é uma credencial real exposta publicamente.

```
VITE_ABACATE_PAY_API_KEY=abc_dev_kqAuKtPpgq5XgUwRNCggq0BQ
```

Impacto: Chave de gateway de pagamento exposta no repositório.

[ALTA] Senha admin commitada no histórico git (Admin.tsx)

■ **Admin.tsx (histórico git)** — linhas commit cdcfff7 / 687d0b2

O histórico git contém commits com ADMIN_PASSWORD hardcoded: "admin123" e depois "9cce22f2". Ambas as senhas são recuperáveis por qualquer pessoa com acesso ao repositório.

```
const ADMIN_PASSWORD = 'admin123'; → '9cce22f2';
```

Impacto: Exposição permanente de senhas no histórico do repositório.

[MÉDIA] Ausência de validação de startup para segredos default

■ **server/index.ts** — linhas 97

O servidor inicia sem verificar se JWT_SECRET e ADMIN_PASSWORD são valores default. Deveria rejeitar iniciar com valores inseguros.

```
const JWT_SECRET = process.env.JWT_SECRET || 'supersecretkey_change_me_in_production';
```

Impacto: Deploy acidental em produção com segredos fracos.

[MÉDIA] URL de checkout Kiwify hardcoded no código-fonte

■ **server/index.ts + services/paymentService.ts** — linhas 383 / 3

A URL de checkout "https://pay.kiwify.com.br/YtpRqSE" está hardcoded no backend e no frontend. Embora não seja um segredo, dificulta rotação e permite identificar o produto.

```
const checkoutUrl = `https://pay.kiwify.com.br/YtpRqSE`;
```

Impacto: Baixo; mas viola princípio de configuração externalizada.

5. Inputs sem Tratamento (XSS)

[ALTA] Injeção de HTML em emails via template literal (nome do cliente)

■ **server/index.ts** — linhas 605, 658

Nas rotas /auto-api/email/send-pdf e /auto-api/email/checkout, o nome do cliente (${name}) é interpolado diretamente no htmlContent do email SEM escape. Um atacante pode usar um nome como "<script>alert(1)</script>" ou "<img src=x onerror=...>" para injetar HTML/JS no corpo do email.

```
<p>Olá <strong>${name}</strong>,</p>
```

Impacto: XSS em clientes de email que renderizam HTML; phishing sofisticado.

[ALTA] Proxy aberto de email — /auto-api/email/send repassa body inteiro

■ **server/index.ts** — linhas 528-545

POST /auto-api/email/send faz JSON.stringify(req.body) e envia direto para a API do Brevo sem autenticação nem sanitização. Qualquer pessoa pode enviar emails arbitrários usando a chave Brevo do projeto.

```
body: JSON.stringify(req.body)
```

Impacto: Abuso da API de email para spam/phishing; consumo da cota Brevo; reputação do domínio.

[MÉDIA] checkoutUrl controlada pelo cliente em email de checkout

■ **server/index.ts** — linhas 649, 662

Na rota /auto-api/email/checkout, o campo checkoutUrl vem do req.body e é inserido diretamente como href no email: href="${checkoutUrl}". Um atacante pode enviar uma URL javascript: ou de phishing que será renderizada no email do cliente.

```
<a href="${checkoutUrl}" ...>Concluir Pagamento</a>
```

Impacto: Phishing via email com URL controlada pelo atacante.

[MÉDIA] Dados pessoais (PII) no data.json legado versionável

■ **server/data.json** — linhas 8-12

data.json contém dados pessoais reais (nome, CPF, email, telefone, placa) de clientes em texto claro. Não está no .gitignore.

```
"customerName": "PEDRO DA COSTA FERREIRA", "customerCpf": "01414397003"
```

Impacto: Violação de LGPD; dados pessoais expostos em arquivo não protegido.

Recomendações Priorizadas

P1 — Imediato: Rotacionar TODAS as chaves de API e segredos. Remover fallbacks hardcoded do JWT_SECRET. Gerar senha admin forte. Limpar histórico git com BFG.

P1 — Imediato: Remover bypass de pagamento via query param e substring de email em produção.

P2 — Curto prazo: Adicionar authMiddleware nas rotas /admin/register-resource e /email/send. Implementar rate-limiting global.

P2 — Curto prazo: Criar helper escapeHtml() e aplicar em todos os templates de email. Validar checkoutUrl contra allowlist.

P2 — Curto prazo: Proteger /resource/recover com verificação de identidade (OTP) e rate-limiting.

P3 — Médio prazo: Adicionar validação de startup que recusa iniciar com segredos default. CI com linter de segredos (gitleaks, truffleHog).

P3 — Médio prazo: Implementar Content-Security-Policy no Nginx. Restringir CORS a origens específicas.

P3 — Médio prazo: Mover data.json com PII para fora do diretório do projeto ou deletar. Adicionar ao .gitignore.

Issues para o GitHub

Abaixo estão as issues formatadas em Markdown, prontas para copiar e colar no GitHub.

--- ISSUE 1 ---

Título: [Segurança] JWT_SECRET com fallback inseguro permite forja de tokens admin

Labels: security, critica

```
## Problema

Em `server/index.ts:97`, o `JWT_SECRET` tem um fallback hardcoded:

```ts
const JWT_SECRET = process.env.JWT_SECRET || 'supersecretkey_change_me_in_production';
```

Se a variável de ambiente `JWT_SECRET` não estiver definida (deploy esquecido, container mal configurado), qualquer pessoa que conheça esse valor pode forjar tokens JWT válidos com role "admin".

Impacto

Acesso total ao painel administrativo: visualização de dados de clientes, alteração de configurações, limpeza de dados.

Sugestão de Correção

1. Remover o fallback: `const JWT_SECRET = process.env.JWT_SECRET;`
2. Adicionar validação na inicialização:

```ts
if (!process.env.JWT_SECRET || process.env.JWT_SECRET.includes('change_me')) {
console.error('FATAL: JWT_SECRET não configurado. Encerrando.');
process.exit(1);
}
```

3. Gerar um segredo forte: `openssl rand -base64 48`

Critérios de Aceite

- [ ] JWT_SECRET sem valor default
- [ ] Servidor recusa iniciar sem JWT_SECRET configurado
- [ ] Segredo em produção tem >= 32 caracteres aleatórios
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Bypass de pagamento via ?dev=true e emails com "test"

Labels: security, alta

```
## Problema

Em `server/index.ts:396-405`, a verificação de pagamento auto-aprova quando:

- `req.query.dev === "true"` (funciona em qualquer ambiente)
```

```
- Email contém "teste", "test" ou "dev"

```ts
const isDev = process.env.APP_MODE === 'dev' ||
req.query.dev === 'true' ||
email.toLowerCase().includes('teste');
```

Impacto
Qualquer pessoa consegue gerar documentos sem pagar.

Sugestão de Correção
1. Condicionar bypass APENAS a `APP_MODE === "dev"`
2. Remover `req.query.dev` e filtros por substring de email
3. Considerar uma allowlist de emails de teste

Critérios de Aceite
- [ ] `?dev=true` não funciona em produção
- [ ] Email com "test" não auto-aprova em produção
- [ ] Teste manual confirma que pagamento real é exigido
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Rotas sensíveis sem autenticação (register-resource, analytics/event, email/send)

Labels: security, alta

```
## Problema
Três rotas aceitam dados de qualquer chamador sem token:

| Rota | Linha | Risco |
|---|---|---|
| `POST /auto-api/admin/register-resource` | 502 | Inserção de recursos falsos |
| `POST /auto-api/analytics/event` | 492 | Injeção de eventos falsos |
| `POST /auto-api/email/send` | 528 | Proxy aberto de email |

## Impacto
- Poluição do dashboard com dados fictícios
- Envio de spam/phishing usando a chave Brevo do projeto
- Consumo de cota de API

## Sugestão de Correção
1. Adicionar `authMiddleware` em register-resource e email/send
2. Implementar rate-limiting ou CAPTCHA em analytics/event
3. Na rota de email, validar campos obrigatórios e restringir destinatários

## Critérios de Aceite
```

```
- [ ] register-resource exige token admin
- [ ] email/send exige token admin ou validação de destinatário
- [ ] analytics/event tem rate-limiting
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Chaves de API e senhas fracas nos arquivos de configuração

Labels: security, critica

```
## Problema
Múltiplas credenciais reais presentes em arquivos no disco:

| Arquivo | Segredo |
|---------|--------|
| `.env:1` | GEMINI_API_KEY (real) |
| `.env:3-5` | DEEPSEEK_API_KEY, BREVO_API_KEY (reais) |
| `.env:7` | ADMIN_PASSWORD=admin123 |
| `.env:8-9` | KIWIFY_CLIENT_ID + SECRET (reais) |
| `.env.example:4` | AbacatePay key real (versionado!) |
| `git history` | Senhas admin hardcoded |

## Sugestão de Correção
1. Rotacionar TODAS as chaves imediatamente
2. Usar gerenciador de segredos (Docker secrets, Vault, etc.)
3. Remover chave real do .env.example
4. Fazer `git filter-branch` ou BFG para limpar histórico
5. Forçar ADMIN_PASSWORD >= 16 chars

## Critérios de Aceite
- [ ] Todas as chaves atuais rotacionadas
- [ ] .env.example contém apenas placeholders
- [ ] Histórico git limpo de credenciais
- [ ] ADMIN_PASSWORD com policy de complexidade
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] XSS e phishing em templates de email — input sem sanitização

Labels: security, alta

```
## Problema
Dados do usuário são interpolados em HTML de email sem escape:

1. **server/index.ts:605** — `${name}` em `<strong>${name}</strong>` (send-pdf)
2. **server/index.ts:658** — `${name}` em email de checkout
```

```
3. **server/index.ts:662** — `${checkoutUrl}` em `href="${checkoutUrl}"` (URL controlada pelo chamador)

## Impacto

Injeção de HTML/JS em emails; links de phishing enviados em nome do AutoRecurso.

## Sugestão de Correção

1. Criar helper de escape HTML:

```ts
function escapeHtml(s: string): string {
return s.replace(/&/g,'&').replace(/</g,'<')
.replace(/>/g,'>').replace(/"/g,'"');
}
```

2. Aplicar em todos os valores interpolados nos templates
3. Validar checkoutUrl contra allowlist de domínios

Critérios de Aceite

- [ ] Todos os campos interpolados em HTML são escaped
- [ ] checkoutUrl validada contra domínios permitidos
- [ ] Teste com payload `<img src=x onerror=alert(1)>` no campo name
```

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Endpoint de busca de recursos expõe dados pessoais (LGPD)

Labels: security, media, lgpd

```
## Problema

POST /auto-api/resource/recover (server/index.ts:512-525) permite buscar recursos de qualquer cliente por CPF parcial (>= 7 dígitos), email ou placa. Não há autenticação, rate-limiting ou CAPTCHA.

## Impacto

Enumeração de dados pessoais protegidos pela LGPD: nomes, CPFs, emails, placas e conteúdo de documentos jurídicos.

## Sugestão de Correção

1. Exigir verificação (código OTP por email/SMS) antes de retornar dados
2. Implementar rate-limiting agressivo (5 req/min por IP)
3. Não retornar dados completos na listagem — apenas confirmar existência

## Critérios de Aceite

- [ ] Busca exige verificação de identidade
- [ ] Rate-limiting implementado
- [ ] Dados sensíveis mascarados na resposta
```

--- FIM ISSUE 6 ---